
MUHAMMAD TAHA

22F-3277

BSC-3C

18/11/2023

PRACTICE LAB

Q1: Take x and y input from user and display message on screen

```
org 0x0100
jmp start
m1: db 'X', 0
m2: db 'Y', 0
msg1: db 'Message 1 printed through taking x and y value input', 0
msg2: db 'Message 2 printed through taking x and y value input', 0
msg3: db 'Message 3 printed through taking x and y value input', 0
len: db 0
clrscr:
    push ax
    push es
    push di
    push cx

    mov ax, 0xb800
    mov es, ax
    mov di, 0
    mov ax, 0x0720
    mov cx, 2000

    cld
    rep stosw

    pop cx
    pop di
    pop es
    pop ax
    ret

strlen:
    push bp
    mov bp, sp
    push es
    push cx
    push di

    les di, [bp+4] ; point es:di to string
    mov cx, 0xffff
```

```
xor al, al
repne scasb
```

```
mov ax, 0xffff
sub ax, cx
dec ax ;
mov [len],ax
```

```
pop di
pop cx
pop es
pop bp
ret 4
```

printstr:

```
push bp
mov bp, sp
push ax
push es
push di
push si
push cx
push ds
```

```
push ds
mov ax, [bp+4]
push ax
call strlen
mov cx, ax          ;length
```

```
cmp cx, 0
jz exit
```

```
mov al, 80
mul byte [bp+8] ;y value
add ax, [bp+10] ;x value
shl ax,1
mov di, ax
```

```
mov ax, 0xb800
mov es, ax
mov si, [bp+4] ; si point to string
```

```
xor ax, ax
mov ah, [bp+6]
```

```
cld
```

l1:

```
lodsb
stosw
loop l1
```

exit:

```
pop ds
pop cx
pop si
pop di
pop es
pop ax
pop bp
ret 8
```

start:

```
call clrscr ; clear the screen
```

```
mov ax, 0          ;x value
push ax
mov ax, 1          ;y value
push ax
mov ax, 0x02
push ax            ;attribute
mov ax, m1
push ax
call printstr
```

```
;taking x input
mov ah, 0 ;
int 0x16
mov ah, 0
sub al, 0x30
push ax ; push x position
```

```
mov ax, 10         ;x value
push ax
mov ax, 1          ;y value
push ax
mov ax, 0x02
push ax            ;attribute
mov ax, m2
push ax
call printstr
```

```
;taking y input
mov ah, 0 ;
int 0x16
mov ah, 0
sub al, 0x30
push ax ; push y position
```

```
mov cl, al
mov ch, 0          ;saving y value to print msg for second string
```

```
mov ax, 1 ; blue on black
push ax ; push attribute
```

```
mov ax, msg1
push ax ; push offset of string
call printstr ; print the string
```

```
;;;;;;;;;print 2nd string;;;;;;;;;
add cx, 1
```

```
mov ax, 0 ;x value
push ax
mov ax, cx ;y value
push ax
mov ax, 0x02
push ax ;attribute
mov ax, m1
push ax
call printstr
```

```
;taking x input
mov ah, 0 ;
int 0x16
mov ah, 0
sub al, 0x30
push ax ; push x position
```

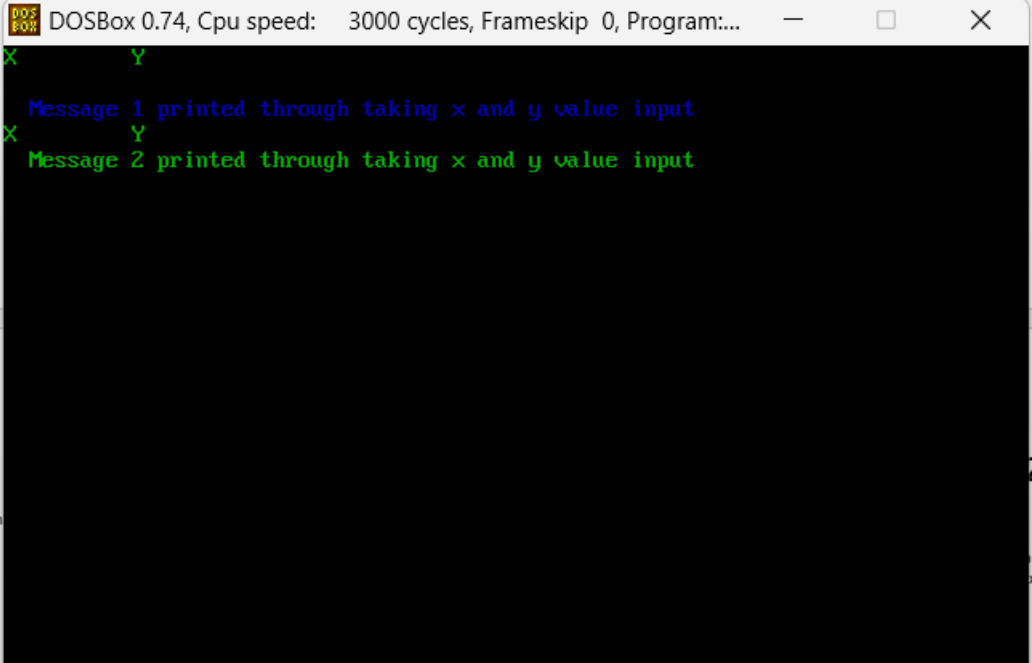
```
mov ax, 10 ;x value
push ax
mov ax, cx ;y value
push ax
mov ax, 0x02
push ax ;attribute
mov ax, m2
push ax
call printstr
```

```
;taking y input
mov ah, 0 ;
int 0x16
mov ah, 0
sub al, 0x30
push ax ; push y position
```

```
mov ax, 2
push ax ; push attribute
```

```
mov ax, msg2
push ax ; push offset of string
call printstr ; print the string
```

```
mov ax, 0x4c00 ; terminate program
int 0x21
```



DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program:...

```
X      Y
Message 1 printed through taking x and y value input
X      Y
Message 2 printed through taking x and y value input
```

Q2: Print string using subroutine with passing register as parameter.

```
org 0x0100
jmp start
message1: db 'This message is printed by subroutine',0
message2: db 'This message is printed through int 0x7 with registers as parameter', 0
len: db 0
clrscr:
    push ax
    push es
    push di
    push cx

    mov ax, 0xb800
    mov es, ax
    mov di, 0
    mov ax, 0x0720
    mov cx, 2000

    cld
    rep stosw

    pop cx
    pop di
    pop es
    pop ax
    ret

strlen:
    push bp
    mov bp,sp
    push es
    push cx
    push di

    les di, [bp+4] ; point es:di to string
    mov cx, 0xffff
    xor al, al
    repne scasb

    mov ax, 0xffff
    sub ax, cx
    dec ax ;
    mov [len],ax

    pop di
    pop cx
    pop es
    pop bp
    ret 4

print:
```

```

push bp
mov bp, sp
push ax
push es
push di
push si
push cx
push ds

push ds
mov ax, [bp+4]
push ax
call strlen
mov cx, ax          ;length

cmp cx, 0
jz exit

mov al, 80
mul byte [bp+8] ;y value
add ax, [bp+10] ;x value
shl ax, 1
mov di, ax

mov ax, 0xb800
mov es, ax
mov si, [bp+4] ; si point to string

xor ax, ax
mov ah, [bp+6]

```

```

cld
l1:
lodsb
stosw
loop l1

```

```

exit:
pop ds
pop cx
pop si
pop di
pop es
pop ax
pop bp
ret 8

```

```

print_with_ISR_07:
push ax
push bx
push cx

```

```
push dx
push si
push di
push bp
push ds
push es
```

```
push cs
pop ds ;point to segment
```

```
push ax      ;x value
push bx      ;y value
push cx      ;attribute
push dx      ;message
call print
```

```
pop es
pop ds
pop bp
pop di
pop si
pop dx
pop cx
pop bx
pop ax
iret
```

start:

```
call clrscr
```

```
mov ax, 4      ;x value
push ax
mov ax, 6      ;y value
push ax
mov ax, 0x01
push ax      ;attribute
mov ax, message1
push ax
call print
```

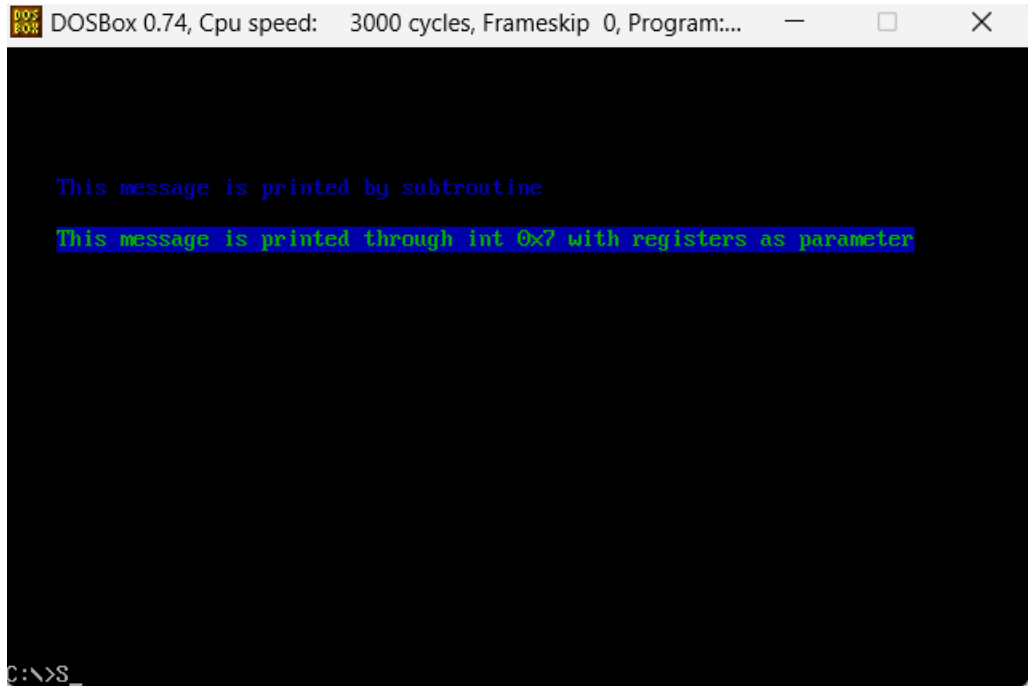
```
mov ax, 0
mov es, ax
```

```
mov ax, 4      ;x value
mov bx, 8      ;y value
mov cx, 0x12   ;attribute
mov dx, message2
```

```
mov word [es:7*4], print_with_ISR_07
mov [es:7*4+2], cs
int 0x7
```



```
mov ax, 0x4c00  
int 0x21
```

A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program:...". The main window area is black with green text. The text "This message is printed by subroutine" is on one line, and "This message is printed through int 0x7 with registers as parameter" is on the next line. At the bottom left, the command prompt shows "C:\>S_".

```
DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program:...  
  
This message is printed by subroutine  
This message is printed through int 0x7 with registers as parameter  
  
C:\>S_
```